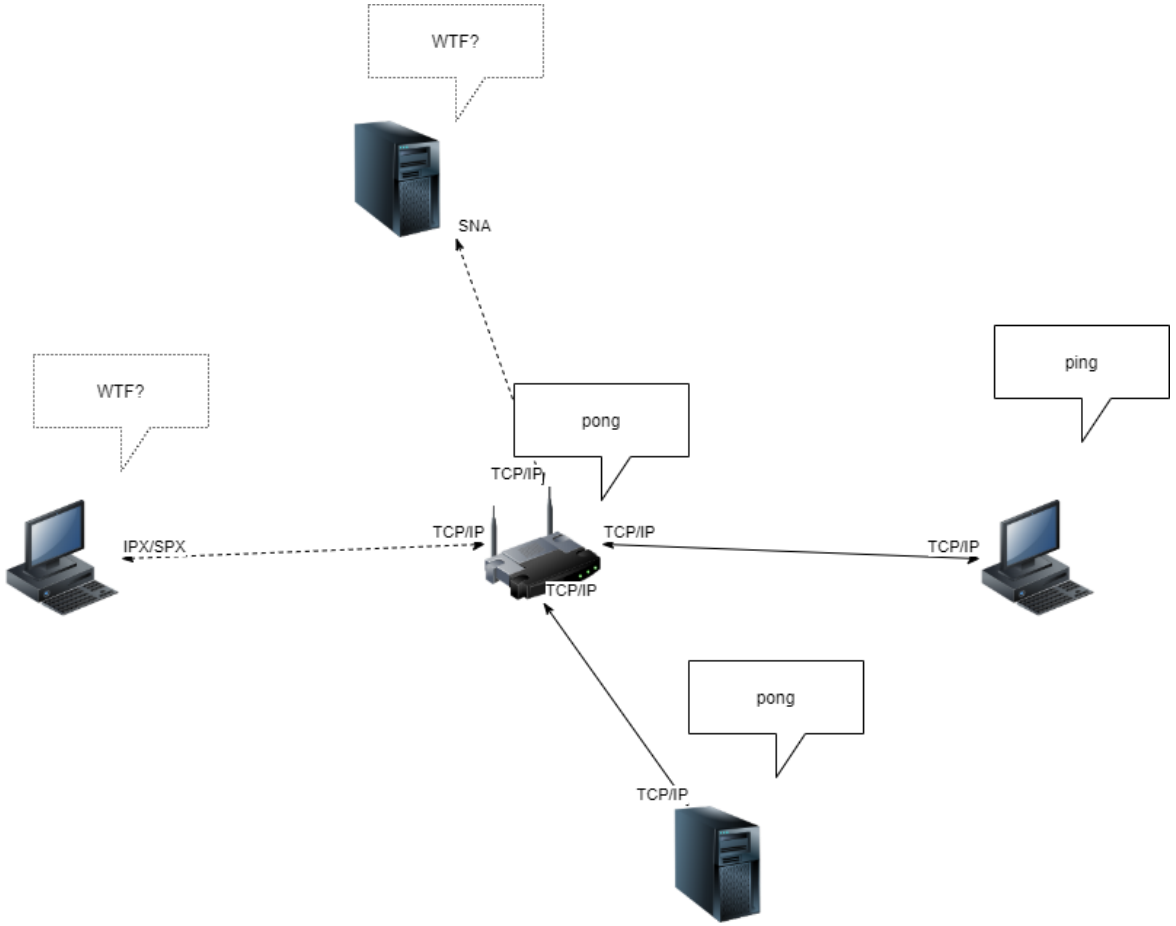
ПЛАМЕН СПАХИЕВ

ФПМИ

МРЕЖОВИ ПРОТОКОЛИ. HTTP. FTP.

МРЕЖОВИ ПРОТОКОЛИ



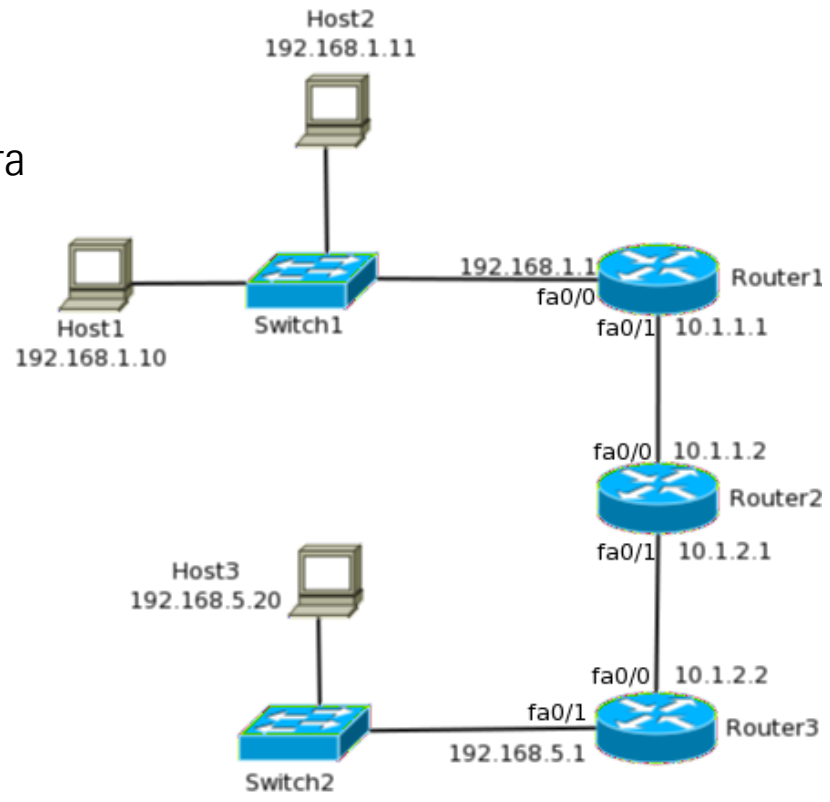
Фиг. 1. Споразумение за комуникация

МРЕЖОВИ ПРОТОКОЛИ

- Рутирани протоколи – могат да бъдат препратени
- Рутиращи протоколи – получават и пренасят информацията

Предимства на динамичното пред статично добавяне на записи в рутиращите таблици:

- Автоматична реакция на промени в мрежата
- Автоматичен избор на най-ефективния маршрут
- Автоматично научаване на нови маршрути без намесата на администратор



Фиг. 2. Рутиране на пакети с данни.

HTTP

Hyper Text Transfer Protocol – използва се за прехвърляне на документи от сървър до браузър с помощта на TCP/IP стака и основната архитектура – клиент/сървър

URL – Uniform Resource Locator

Възможности

- Работи в приложния слой;
- Сървърът не запазва историята на HTTP сесиите;
- Осигурява двупосочен трансфер на данни;
- Използва т.нар. съгласуване на способността (capability negotiation);
- Поддържа кеширане;
- HTTP дава възможност за поддържане на посредници (proxy).

HTTP

Компоненти

- Браузър (browser);
- Комуникационни вериги;
- HTTP сървър;
- Прокси;
- Потребителски агенти (User Agents);
- Ресурси;
- Тунел.

HTTP СЕСИИ

- Клиентът осъществява TCP връзка със сървъра;
- Клиентът изпраща своето запитване и очаква отговор;
- Сървърът обработва запитването и изпраща своя отговор, заедно с код за статус.

URI = scheme ":" ["//" authority] path ["?" query] ["#" fragment],
където
authority = [userinfo "@"] host [":" port]

Така например напълно валидни URL са:

http://myhost.com/path/to/resource.html
http://myhost.com:8080/resource.html?query=giveittome
http://myhost.com:8080/path/?query=giveittome#fromline5
http://username:password@myhost.com:8080/specific/hierarchical/path?query=giveittome#startingfromline5

HTTP СЕСИИ

```
GET / HTTP/1.1
Host: developer.mozilla.org
Accept-Language: fr
```

Фиг. 3. Пример за HTTP request

```
POST /contact_form.php HTTP/1.1
Host: developer.mozilla.org
Content-Length: 64
Content-Type: application/x-www-form-urlencoded

name=Joe%20User&request=Send%20me%20one%20of%20your%20catalogue
```

Фиг. 4. Пример за HTTP request

HTTP СЕСИИ

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 55743
Connection: keep-alive
Cache-Control: s-maxage=300, public, max-age=0
Content-Language: en-US
Date: Thu, 06 Dec 2018 17:37:18 GMT
ETag: "2e77ad1dc6ab0b53a2996dfd4653c1c3"
Server: meinheld/0.6.1
Strict-Transport-Security: max-age=63072000
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Vary: Accept-Encoding,Cookie
Age: 7

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>A simple webpage</title>
</head>
<body>
  <h1>Simple HTML5 webpage</h1>
  <p>Hello, world!</p>
</body>
</html>
```

Фиг. 4. Пример за HTTP Server Response

HTTP СЕСИИ

Name	Description
A-IM	Acceptable instance-manipulations for the request.
Accept	Media type(s) that is/are acceptable for the response.
Accept-Charset	Character sets that are acceptable.
Accept-Datetime	Acceptable version in time.
Accept-Encoding	List of acceptable encodings. See HTTP compression.
Accept-Language	List of acceptable human languages for response. See Content negotiation.
Access-Control-Request-Method	Initiates a request for cross-origin resource sharing with Origin (below).
Access-Control-Request-Headers	
Authorization	Authentication credentials for HTTP authentication.
Cache-Control	Used to specify directives that *must* be obeyed by all caching mechanisms along the request-response chain.
Connection	Control options for the current connection and list of hop-by-hop request fields. Must not be used with HTTP/2.
Content-Encoding	The type of encoding used on the data. See HTTP compression.
Content-Length	The length of the request body in octets (8-bit bytes).
Content-MD5	A Base64-encoded binary MD5 sum of the content of the request body.
Content-Type	The Media type of the body of the request (used with POST and PUT requests).
Cookie	An HTTP cookie previously sent by the server with `Set-Cookie`
Date	The date and time at which the message was originated (in "HTTP-date" format as defined by RFC 7231 Date/Time Formats).
Expect	Indicates that particular server behaviors are required by the client.
Forwarded	Disclose original information of a client connecting to a web server through an HTTP proxy.
From	The email address of the user making the request.
Host	The domain name of the server (for virtual hosting), and the TCP port number on which the server is listening. The port number may be omitted if the port is the standard port for the service requested. Mandatory since HTTP/1.1 If the request is generated directly in HTTP/2, it should not be used.
HTTP2-Settings	A request that upgrades from HTTP/1.1 to HTTP/2 MUST include exactly one `HTTP2-Setting` header field. The `HTTP2-Settings` header field is a connection-specific header field that includes parameters that govern the HTTP/2 connection, provided in anticipation of the server accepting the request to upgrade.
If-Match	Only perform the action if the client supplied entity matches the same entity on the server. This is mainly for methods like PUT to only update a resource if it has not been modified since the user last updated it.
If-Modified-Since	Allows a *304 Not Modified* to be returned if content is unchanged.
If-None-Match	Allows a 304 Not Modified to be returned if content is unchanged, see HTTP ETag.
If-Range	If the entity is unchanged, send me the part(s) that I am missing; otherwise, send me the entire new entity.
If-Unmodified-Since	Only send the response if the entity has not been modified since a specific time.
Max-Forwards	Limit the number of times the message can be forwarded through proxies or gateways.
Origin	Initiates a request for cross-origin resource sharing (asks server for Access-Control-* response fields).
Pragma	Implementation-specific fields that may have various effects anywhere along the request-response chain.
Prefer	Allows client to request that certain behaviors be employed by a server while processing a request.
Proxy-Authorization	Authorization credentials for connecting to a proxy.
Range	Request only part of an entity. Bytes are numbered from 0. See Byte serving.
Referer	This is the address of the previous web page from which a link to the currently requested page was followed. (The word "referrer" has been misspelled in the RFC as well as in most implementations to the point that it has become standard usage and is considered correct terminology)
TE	The transfer encodings the user agent is willing to accept: the same values as for the response header field Transfer-Encoding can be used, plus the "trailers" value (related to the "chunked" transfer method) to notify the server it expects to receive additional fields in the trailer after the last, zero-sized, chunk. Only trailers is supported in HTTP/2
Trailer	The Trailer general field value indicates that the given set of header fields is present in the trailer of a message encoded with chunked transfer coding.
Transfer-Encoding	The form of encoding used to safely transfer the entity to the user. Currently defined methods are: chunked, compress, deflate, gzip, identity. Must not be used with HTTP/2.
User-Agent	The user agent string of the user agent.
Upgrade	Ask the server to upgrade to another protocol. Must not be used in HTTP/2.
Via	Informs the server of proxies through which the request was sent.
Warning	A general warning about possible problems with the entity body.

Табл. 1. Списък с използваните HTTP Request Headers

HTTP ***СЕСИИ***

Метод	Значение
GET	Изисква определен ресурс. GET requests само изискват данни
HEAD	Идентичен на GET, но се пропускат реалните данни
POST	Използва се за изпращане на данни, често с резултат промяна на статус или други странични ефекти
PUT	Заменя всички стойности на ресурса с тези от пейлоуда на заявката
DELETE	Изтрива специфичния ресурс
CONNECT	Изгражда тунел към сървъра
OPTIONS	Описва опциите за комуникация
TRACE	Използва се за тестване на пътя към ресурса
PATCH	Прилага частични модификации към ресурса

Табл. 3. Списък с HTTP методите

HTTP ***СЕСИИ***

Кодове за състояние в HTTP

- 1xx: Информационно. Запитването е прието, процесът продължава;
- 2xx: Успешно. Действието е получено успешно, разбрано е и е прието;
- 3xx: Пренасочване. Трябва да бъде предприето допълнително действие за изпълнение на запитването;
- 4xx: Грешка, свързана с клиента. Запитването съдържа неправилен синтаксис и не може да бъде изпълнено;
- 5xx: Грешка, свързана със сървъра. Сървърът не е успял да изпълни запитването.

HTTP СЕСИИ

Код на състояние	Значение
100	Продължение
101	Превключване на протоколи
200	OK
201	Създадено
202	Прието
203	Неавтентична информация
204	Няма съдържание
205	Нулиране на съдържание
206	Частично съдържание
300	Множество избори
301	Преместено постоянно
302	Преместено временно
303	Вижте друго
304	Не е модифицирано
305	Използвай прокси
400	Грешно запитване
401	Неавторизирано
402	Необходимо е заплащане
403	Забранено
404	Не е намерено
405	Методът не е разрешен
406	Неприемливо
407	Необходима е автентикация на прокси
408	Таймаут на запитванията
409	Конфликт
410	Заминало
411	Изисква дължина
412	Предварително условие не е изпълнено
413	Единицата на запитване е прекалено голяма
414	URL на запитването е прекалено голям
415	Типът на преносната среда не се поддържа
500	Вътрешна грешка на сървър
501	Неимпленментирано
502	Грешен шлюз
503	Недостъпна услуга
504	Таймаут на шлюз
505	Неподдържана версия на HTTP

Табл. 4. Списък с кодовете на HTTP

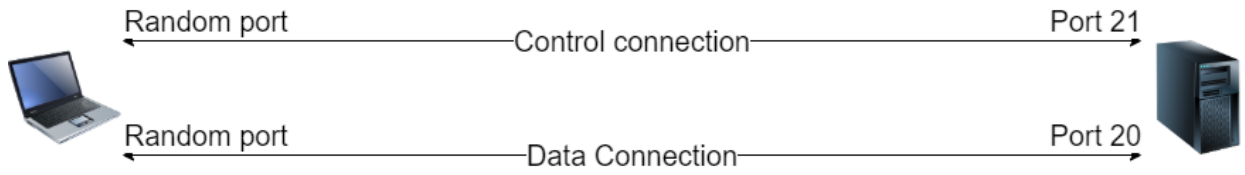
FTP

File Transfer Protocol – отговаря за по-голямата част от трансфера на файлове в компютърните мрежи и в частност - Интернет

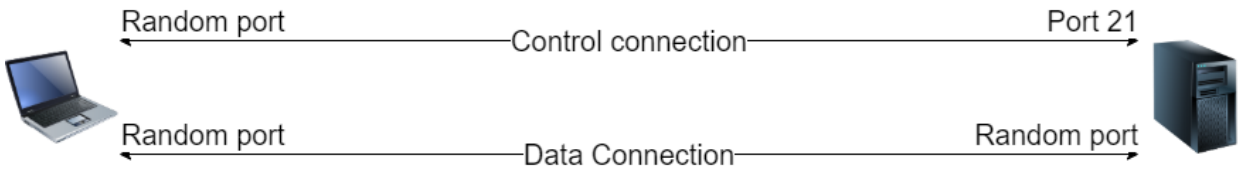
Елементи на FTP

- Потребителски интерфейс;
- Клиентски протоколен интерпретатор;
- Сървърен протоколен интерпретатор;
- Клиентски процес на трансфер на данни;
- Сървърен процес за трансфер на данни.

FTP



Фиг. 5. Активна FTP връзка между клиент и сървър



Фиг. 6. Пасивна FTP връзка между клиент и сървър

FTP КОМАНДИ

Команда и аргумент	Описание
ACCT[account info]	Идентифицира потребителски акаунт
CDUP	Промяна към горна директория
CWD[path]	Промяна на работната директория
PASS[password]	Парола за потребител
QUIT	Прекъсва връзката
REIN	Реинициализиране на връзката
SMNT[path]	Монтиране на структура
USER[username]	Име на потребител на сървъра

Табл. 5. Команди за контрол на достъпа

FTP КОМАНДИ

Команда и аргумент	Описание
MODE[mode]	Режим на предаване
PASV	Указание за пасивна връзка
PORT[host port]	Задава номер на клиентския порт
STRU[structure]	Файлова структура
TYPE[type]	Файлов тип

Табл. 6. Команди за параметризиране на трансфера

FTP КОМАНДИ

Команда и аргумент	Описание
ABOR	Прекратява предишна команда
ALLO[bytes]	Заделя пространство за файл преди изпращането му
APPE[path name]	Прибавя файла
DELE[path name]	Изтриване на файл
HELP[string]	Помощ за конкретна команда
LIST[path]	Изпраща списък с файловете
MKD[path]	Създава директория
NLST[path]	Списък с имена
NOOP	Няма действие
PWD	Отпечатва работната директория
REST[marker]	Рестартира трансфера
RETR[path]	Извличане на файл
RMD[path]	Премахване на директория
RNFR[path]	Преименува от
RNTO[path]	Преименуване на
SITE[string]	Параметри на сайт
STAT[path]	Състояние
STOR[file]	Съхраняване на файла на сървъра
STOU	Подобна на STOP. Презаписва съществуващия файл
SYST	Отчита операционната система

Табл. 7. Команди за контрол на сървъра

FTP ОТГОВОРИ

Range	Purpose
1xx	Positive Preliminary reply
	The requested action is being initiated; expect another reply before proceeding with a new command. (The user-process sending another command before the completion reply would be in violation of protocol; but server-FTP processes should queue any commands that arrive while a preceding command is in progress.) This type of reply can be used to indicate that the command was accepted and the user-process may now pay attention to the data connections, for implementations where simultaneous monitoring is difficult. The server-FTP process may send at most, one 1xx reply per command.
2xx	Positive Completion reply
	The requested action has been successfully completed. A new request may be initiated.
3xx	Positive Intermediate reply
	The command has been accepted, but the requested action is being held in abeyance, pending receipt of further information. The user should send another command specifying this information. This reply is used in command sequence groups.
4xx	Transient Negative Completion reply
	The command was not accepted and the requested action did not take place, but the error condition is temporary and the action may be requested again. The user should return to the beginning of the command sequence, if any. It is difficult to assign a meaning to "transient", particularly when two distinct sites (Server- and User-processes) have to agree on the interpretation. Each reply in the 4xx category might have a slightly different time value, but the intent is that the user-process is encouraged to try again. A rule of thumb in determining if a reply fits into the 4xx or the 5xx (Permanent Negative) category is that replies are 4xx if the commands can be repeated without any change in command form or in properties of the User or Server (e.g., the command is spelled the same with the same arguments used; the user does not change his file access or user name; the server does not put up a new implementation.)
5xx	Permanent Negative Completion reply
	The command was not accepted and the requested action did not take place. The User-process is discouraged from repeating the exact request (in the same sequence). Even some "permanent" error conditions can be corrected, so the human user may want to direct his User-process to reinitiate the command sequence by direct action at some point in the future (e.g., after the spelling has been changed, or the user has altered his directory status.)
6xx	Protected reply
	The RFC 2228 introduced the concept of protected replies to increase security over the FTP communications. The 6xx replies are Base64 encoded protected messages that serves as responses to secure commands. When properly decoded, these replies fall into the above categories.

Табл. 8. Значение на първата цифра от кода на отговора

FTP ОТГОВОРИ

Range	Purpose
x0x	Syntax
	These replies refer to syntax errors, syntactically correct commands that don't fit any functional category, unimplemented or superfluous commands.
x1x	Information
	These are replies to requests for information, such as status or help.
x2x	Connections
	Replies referring to the control and data connections.
x3x	Authentication and accounting
	Replies for the login process and accounting procedures.
x4x	Unspecified as of RFC 959.
x5x	File system
	These replies indicate the status of the Server file system vis-a-vis the requested transfer or other file system action.

Табл. 9. Значение на втората цифра от кода на отговора